AN0040-EC NB-IoT OpenCPU Development Guide book_V1.0

EigenCOMM Wireless Microcontroller

Description

This guidebook tries to help developer who will develop their own application feature on EC NB-IoT chipset. Then external MCU could be saved.

目录

1. Forward

1.1 Purpose

The EC NB-IoT chipset could be used as an application processor to implement customer application directly on it. This guidebook will introduce the hardware resource, software architecture, module, API and etc to help customer's development.

1.2 Develop Procedure

1） Get Eigencomm SDK

2） Understand the OpenCPU architecture

3） Analysis application requirement and get the solution

4） Evaluate solution and implement the detail

2. OpenCPU Overview

2.1 Architecture

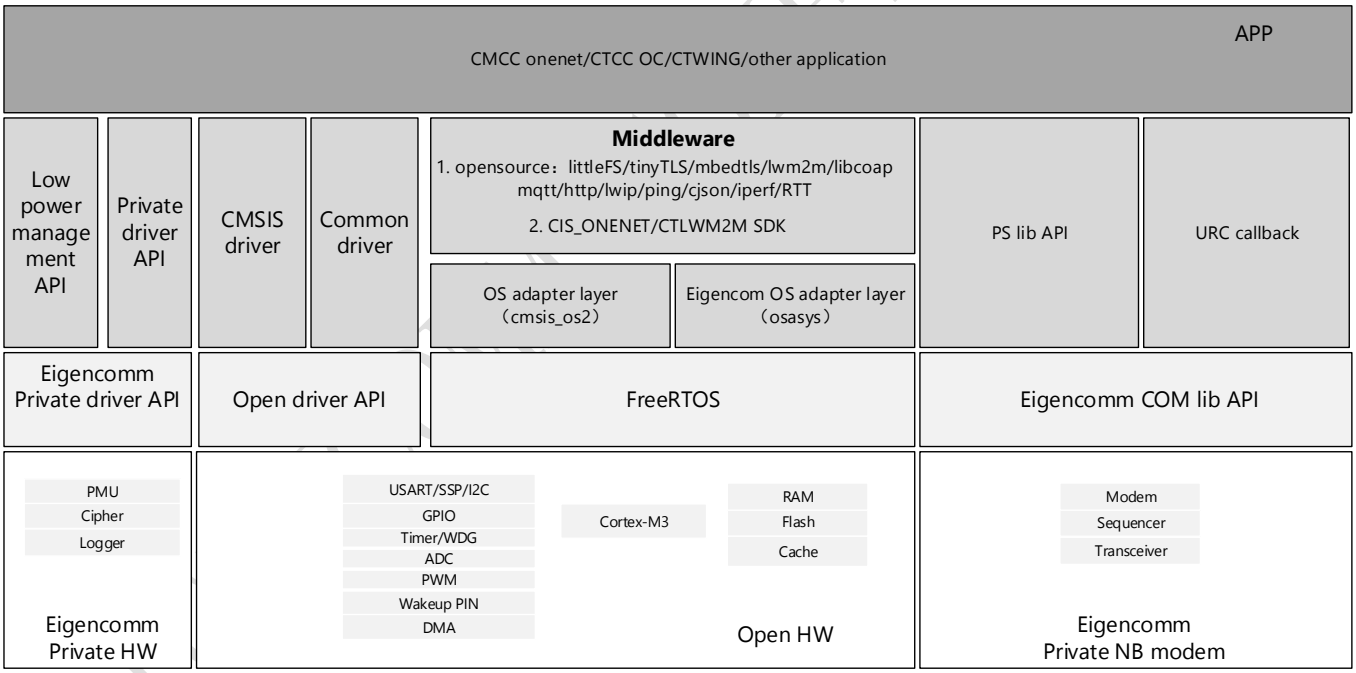

Figure 1 OpenCPU architecture

2.2 OpenCPU Resource

2.2.1 Processor

32bit ARM Cortex-M3 processor working at 204MHz

2.2.2 Memory

- 256KB+16KB SRAM and 4MB NorFlash

Memory reserved for user：

- 1MB Flash
- SRAM：HEAP will occupy all the left space of the main RW/ZI data region, which means HEAP will have dynamic size.

  The scatter file will check the HEAP size to make sure at least 56KB space is for HEAP. User could add their global variable as needed, but need to make sure the size of the global variable is reasonable.

2.3 OpenCPU Compile

For detail, please refer to AN0020-EC NB-IoT SDK Compiling Guidebook.

For example, modify the project's makefile as below means disable AT module.

```
AVAILABLE_TARGETS = ec616_0h00 ec616z0_0h00
TOOLCHAIN         = ARMCC
BINNAME           = onenet-demo-flash
TOP               := ../../../../../..

BUILD_AT                    = n
BUILD_AT_DEBUG              = n
THIRDPARTY_CISONENET_ENABLE = y
THIRDPARTY_WAKAAMA_ENABLE   = n

CFLAGS_INC        +=  -I ../inc
obj-y             += PLAT/project/$(TARGET)/apps/onenet_example/src/app.o \
                     PLAT/project/$(TARGET)/apps/onenet_example/src/cisAsynEntry.o \
        PLAT/project/$(TARGET)/apps/onenet_example/src/bsp_custom.o


include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

Figure 2 OpenCPU Compile

2.4 OpenCPU Data Type

Suggested to include commontypedef.h, and use the basic data type definition in it.

2.5 OpenCPU Limitation

1. Interrupt handler running time should be less than 50us.
   disable interrupt time limitation: less than 100us per 1ms.
2. Task priority limitation
   Application task's priority should below osPriorityNormal(24)
3. When PMU is enabled, and allow to enter deeper sleep mode than SLEEP1, OS timer will be invalid.
   HIB timer API is provided for timer requirement in deep sleep mode.

3. OpenCPU API

3.1 System API

Two types system API are provided in SDK. One is follow ARM CMSIS-OS2 standard, another one is EC defined API.

For detail of CMSIS-OS2, please refer to official document. This section will introduce some EC defined API.

3.1.1 String copy

- **header file**

  #include " osasys.h"

- **prototype**

  CHAR strdup(const CHAR*string)*

- **parameter**

  string: original string

- **return**

  new copy of original string

- **description**

  POSIX style string copy API, mainly used in third party module.

3.1.2 assert

- **header file**

  #include " os_exception.h" 0r #include " assert.h"

- **prototype**

  EC_ASSERT(x,v1,v2,v3)

  assert(x)

- **parameter**

  x: assert condition, trigger assert if false

  v/v2/v3: 3 output parameter when assert triggered

- **return**

  N/A

- **description**

  two assert APIs are provided in EC SDK. One is EC_ASSERT(),"os_exception.h" should be included.

  When assert triggered via this API, arm core register will be print in unilog, and will trigger ram dump, silent reset per setting. Another one is simple assert() in C lib, "assert.h" should be included, this API is mainly used by third party open source module. For user develop, EC_ASSERT() is suggested.

3.1.3 OSA signal

OS message queue adapter Defined by EC.

Signal create API:

*void OsaCreateSignal (UINT16 sigId, UINT16 sigBodySize, SignalBuf **signal)*

*void OsaCreateZeroSignal (UINT16 sigId, UINT16 sigBodySize, SignalBuf **signal)*

*void OsaCreateFastSignal (UINT16 sigId, UINT16 sigBodySize, SignalBuf **signal)*

the difference between OsaCreateSignal()与 OsaCreateZeroSignal() is whether to set the sigBody to 0. Malloc is called in above 2 APIs, so should not be called in interrupt context.

Then OsaCreateFastSignal() is defined to be called in interrupt context.

Send signal API：

*void OsaSendSignal(UINT16 taskId, SignalBuf **signal)*

*void OsaSendDumpSignal(UINT16 taskId, SignalBuf **signal)*

*void OsaSendNoDumpSignal(UINT16 taskId, SignalBuf **signal)*

the difference of above 3 APIs is mainly for debug purpose. OsaSendSignal and OsaSendDumpSignal is similar , they will dump signal content to unilog. OsaSendNoDumpSignal will not dump the signal content.

Receive signal API：

*void OsaReceiveSignal(UINT16 taskId, SignalBuf **signal)*

Destroy signal API：

*void OsaDestroySignal (SignalBuf **signal)*

*void OsaDestroyFastSignal (SignalBuf **signal)*

each matched to create APIs.

3.1.4 Dynamic memory management

- **header file**

  #include " cmsis_os2.h"

- **prototype**

  void malloc(size_t size)*

  void realloc(void *pv, size_t xWantedSize)*

  void calloc(size_t n, size_t Size)*

  void free(void *p)*

- **parameter**

  N/A

- **return**

  when malloc，return NULL means allocate fail；if allocate success return valid pointer.

3.2 Driver API

Please refer to“EC61x_API_Reference_Manual”.

3.3 Log API

EC SDK support traditional printf API, but it is not efficient and not suggest to use, since it will affect the timing of NB modem and protocol stack. EC SDK provides UNILOG API, the API is efficient, could be output to PC via UART, and EC provide EPAT tool to parse and capture the log.

3.3.1 Set unilog debug level

- **header file**

  #include " debug_trace.h"

- **prototype**

  void uniLogDebugLevelSet (debugTraceLevelType debugLevel)

- **parameter**

  debugLevel: LOG level

  support 6 levels:P_DEBUG/P_INFO/P_VALUE/P_SIG/P_WARNING/P_ERROR

- **return**

  N/A

3.3.2 Unilog API(macro)

- **header file**

  #include " debug_trace.h"

- **prototype**

  ECOMM_TRACE(moduleID, subID, debugLevel, argLen, format, ...)

- **parameter**

  moduleID: LOG module ID

  subID: subID under same moduleID ,separate logs in the same module

  debugLevel: debug level

  argLen: number of parameter

  format： format information

  ... : variable length parameter

  NOTE: current only support：%d,%x,%f,%u。

- **return**

  N/A

3.3.3 Unilog dump API

- **header file**

  #include " debug_trace.h"

- **prototype**

  ECOMM_DUMP(moduleID, subID, debugLevel, format, dumpLen, dump)

- **parameter**

  moduleID: LOG module ID

  subID: subID under same moduleID ,separate logs in the same module

  debugLevel: debug level

  format： format information

  dumpLen: dump data length

  dump: dump data

- **return**

  N/A

  Note: the dump data length should not too long. If length is greater than 200Bytes, log may lost.

3.3.4 Unilog enhanced dump API

- **header file**

  #include " debug_trace.h"

- **prototype**

  ECOMM_DUMP_POLLING(moduleID, subID, debugLevel, format, dumpLen, dump)

- **parameter**

  moduleID: LOG module ID

  subID: subID under same moduleID ,separate logs in the same module

  debugLevel: debug level

  format： format information

  dumpLen: dump data length

  dump: dump data

- **return**

  N/A

  This API will loop the log FIFO space before send the log data, more reliable then ECOMM_DUMP API.

  But it is suggest not to dump too much data, since it will delay the caller task.

3.3.5 Unilog string print API

- **header file**

  #include " debug_trace.h"

- **prototype**

  ECOMM_STRING(moduleID, subID, debugLevel, format, string)

- **parameter**

  moduleID: LOG module ID

  subID: subID under same moduleID ,separate logs in the same module

  debugLevel: debug level

  format： format information

  string: string tobe printed

- **return**

  N/A

3.4 Power API

Please refer to“AN0024-EC NB-IoT PMU Development Guidebook_V1.4.pdf”。

3.5 Filesystem API

LittleFS is implemented in SDK, LFS API is listed below. For more detail user could refer to LittleFS official introduction.

3.5.1 Open file

- **header file**

  #include " lfs_port.h"

- **prototype**

  *int LFS_FileOpen(lfs_file_t *file, const char *path, int flags)*

- **parameter**

  file: file handle

  path: file name

flags: LFS_O_RDONLY/ LFS_O_WRONLY/ LFS_O_RDWR/ LFS_O_CREAT

- **return**

  a negative error code on failure

3.5.2 Read file

- **header file**

  #include " lfs_port.h"

- **prototype**

  *lfs_ssize_t LFS_FileRead(lfs_file_t *file, void *buffer, lfs_size_t size)*

- **parameter**

  file: file handle

  buffer: read buffer

  size: size of bytes of data to be read

- **return**

  number of bytes read or a negative error code on failure

3.5.3 Write file

- **header file**

  #include " lfs_port.h"

- **prototype**

  *lfs_ssize_t LFS_FileWrite(lfs_file_t *file, void *buffer, lfs_size_t size)*

- **parameter**

  file: 文件句柄

  buffer: write buffer

  size: size of bytes of data to be write

- **return**

  number of bytes written or a negative error code on failure

3.5.4 Close file

- **header file**

  #include " lfs_port.h"

- **prototype**

  *int LFS_FileClose(lfs_file_t *file)*

- **parameter**

  file: file handle

- **return**

  a negative error code on failure

3.5.5 Delete file

- **header file**

  #include " lfs_port.h"

- **prototype**

  *int LFS_Remove(const char *path)*

- **parameter**

  path: file name to be deleted

- **return**

  a negative error code on failure

3.5.6 Reset file pointer

- **header file**

  #include " lfs_port.h"

- **prototype**

  *int LFS_FileRewind(lfs_file_t *file)*

- **parameter**

  file: file handle

- **return**

  a negative error code on failure

3.5.7 Change position of the file

- **header file**

  #include " lfs_port.h"

- **prototype**

  *lfs_soff_t LFS_FileSeek(lfs_file_t *file, lfs_soff_t off, int whence)*

- **parameter**

  file: file handle

  off: the number of bytes to offsets from whence

  whence: the position from where offset is added

  LFS_SEEK_SET/LFS_SEEK_CUR/LFS_SEEK_END

- **return**

  a negative error code on failure

3.5.8 Get file size

- **header file**

  #include " lfs_port.h"

- **prototype**

  *lfs_soff_t LFS_FileSize (lfs_file_t *file)*

- **parameter**

  file: file handle

- **return**

  file size

3.6 Socket API

LWIP is ported in the SDK. Some socket APIs are listed below, for more information please refer to LWIP official document.

3.6.1 Create socket

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  int socket (int domain, int type, int protocol)

- **parameter**

  domain: AF_INET

  type: support 3 types SOCK_STREAM/SOCK_DGRAM/SOCK_RAW match to TCP/UDP/RAW

  protocol: IPPROTO_TCP if TCP ； IPPROTO_UDP if UDP

- **return**

  a negative error code on failure or socket handle

3.6.2 Set socket options

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int setsockopt(int s, int level, int optname, const void *optval, socklen_t optlen)*

- **parameter**

  s: socket handle

  level/optname/optval/optlen: refer to LWIP description

- **return**

  a negative error code on failure

3.6.3 Socket connect

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int connect(int s, const struct sockaddr *name, socklen_t namelen)*

- **parameter**

  s: local socket handle

  name: sockaddr information of socket connection

  namelen：sockaddr length

- **return**

  a negative error code on failure

3.6.4 Socket send

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int send(int s, const void *data, size_t size, int flags)*

  *int sendto(int s, const void *data, size_t size, int flags, const struct sockaddr *to, socklen_t tolen)*

- **parameter**

  s: local socket handle

data: data to be sent

size：send data size

flags：default is 0，for other setting please refer to lwip API description

to：destination socket address

tolen：size of destination socket address

- **return**

  a negative error code on failure or number of bytes sent

3.6.5 Socket receive

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int recv(int s, void *mem, size_t len, int flags)*

  *int recvfrom(int s, void *mem, size_t len, int flags, struct sockaddr *from, socklen_t *fromlen)*

- **parameter**

  s: local socket handle

  mem: the receive buffer

  size：number of bytes to be received

  flags：default is 0，for other value please refer to lwip API description

  from：receive address

  fromlen：size of receive address

- **return**

  a negative error code on failure or number of bytes received

3.6.6 Select

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int select(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset,*
  *struct timeval *timeout)*

- **parameter**

- **return**

  please refer to lwip API description

3.6.7 Socket close

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  int closesocket(int s)

- **parameter**

  s: socket handle to be closed

- **return**

  a negative error code on failure

3.6.8 TCP send API with RAI feature

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int ps_send(int s, const void *data, size_t size, int flags, u8_t dataRai,*
  bool exceptdata)

- **parameter**

  s: socket handle

  data: date to be sent

  size: number of bytes to be sent

  flags:

  / Flags we can use with send and recv. */*

  #define MSG_PEEK 0x01 / Peeks at an incoming message */*

  #define MSG_WAITALL 0x02 / Unimplemented: Requests that the function block until the full amount of data requested can be returned */*

  #define MSG_OOB 0x04 / Unimplemented: Requests out-of-band data. The significance and semantics of out-of-band data are protocol-specific */*

  #define MSG_DONTWAIT 0x08 / Nonblocking i/o for this operation only */*

  #define MSG_MORE 0x10 / Sender will send more */*

  dataRai：请参考以下定义：

  / SOCKET DATA RAI info: Release assistant info */*

  #define PS_SOCK_RAI_NO_INFO 0

  #define PS_SOCK_RAI_NO_UL_DL_FOLLOWED 1

  #define PS_SOCK_ONLY_DL_FOLLOWED 2

  Exceptdata： usually set to false

- **return**

  a negative error code on failure or number of bytes sent

3.6.9 UDP send API with RAI feature

- **header file**

  #include " lwip/sockets.h"

- **prototype**

  *int int ps_sendto(int s, const void *data, size_t size, int flags,*
  *const struct sockaddr *to, socklen_t tolen, u8_t dataRai,*
  bool exceptdata)

- **parameter**

  s: socket handle

  data: data to be sent

  size: number of bytes to be sent

  flags:

  / Flags we can use with send and recv. */*

  #define MSG_PEEK 0x01 / Peeks at an incoming message */*

  #define MSG_WAITALL 0x02 / Unimplemented: Requests that the function block until the full amount of data requested can be returned */*

#define MSG_OOB 0x04 / Unimplemented: Requests out-of-band data. The significance and semantics of out-of-band data are protocol-specific */*

#define MSG_DONTWAIT 0x08 / Nonblocking i/o for this operation only */*

#define MSG_MORE 0x10 / Sender will send more */*

to：destination address

tolen：length of destination address

dataRai：

/ SOCKET DATA RAI info: Release assistant info */*

#define PS_SOCK_RAI_NO_INFO 0

#define PS_SOCK_RAI_NO_UL_DL_FOLLOWED 1

#define PS_SOCK_ONLY_DL_FOLLOWED 2

Exceptdata**:** usually set to false

- **return**

  a negative error code on failure or number of bytes sent

3.7 Protocol Stack API

This section introduce the NB protocol stack API, most the API is synchronize API.

3.7.1 Register PS event callback

- **header file**

  #include " eventcallback.h"

- **prototype**

  NBStatus_t registerPSEventCallback(groupMask_t groupMask, psEventCallback_t callback)

- **parameter**

  groupMask: filter the Event by groups, Group is defined as below：

```
typedef enum {
    NB_GROUP_BASE_MASK = (0X01 << CAC_BASE),
    NB_GROUP_DEV_MASK  = (0X01 << CAC_DEV),
    NB_GROUP_MM_MASK   = (0X01 << CAC_MM),
    NB_GROUP_PS_MASK   = (0X01 << CAC_PS),
    NB_GROUP_SIM_MASK  = (0X01 << CAC_SIM),
    NB_GROUP_SMS_MASK  = (0X01 << CAC_SMS),
    NB_GROUP_ALL_MASK  = 0X7FFFFFFF
} groupMask_t;
```

callback：PS EVENT callback to be registered, the callback will be triggered when related PS EVENT happened.

Event ID	Description
NB_URC_ID_SIM_READY	SIM success detected event
NB_URC_ID_SIM_REMOVED	SIM removed event
NB_URC_ID_PS_BEARER_ACTED	Default BEARER enable event
NB_URC_ID_PS_BEARER_DEACTED	Default BEARER disable event

NB_URC_ID_PS_CEREG_CHANGED	network registered state changed event (including CellID info)
NB_URC_ID_PS_NETINFO	Network connection state changed event
NB_URC_ID_MM_SIGQ	Signal strength changed event
NB_URC_ID_MM_TAU_EXPIRED	TAU timer expired event
NB_URC_ID_MM_BLOCK_STATE_CHANGED	Network blocked state changed event
NB_URC_ID_MM_NITZ_REPORT	Network UTC report event
NB_URC_ID_MM_CERES_CHANGED	Network celelvel changed event

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.2 Deregister PS callback event

- **header file**

  #include " eventcallback.h"

- **prototype**

  NBStatus_t deregisterPSEventCallback(psEventCallback_t callback)

- **parameter**

  Callback：PS EVENT callback to be deregistered

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.3 Trigger PS event

- **header file**

  #include " eventcallback.h"

- **prototype**

  *void PSTriggerEvent(urcID_t eventID, void *param, UINT32 paramLen)*

- **parameter**

  eventID： PS Event ID

  *param： the message pointer of this event

  ParamLen：the message length of this event

- **return**

  N/A

3.7.4 Get IMSI

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetImsiNumSync(CHAR *imsi)*

- **parameter**

  *imsi： a buffer pointer to store returned imsi

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.5 Get ICCID

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetIccidNumSync(CHAR *iccid)*

- **parameter**

  ***** iccid： a buffer pointer to store returned iccid

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.6 Get IMEI

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appGetImeiNumSync(CHAR imei)*

- **parameter**

  *****imei： a buffer pointer to store returned imei

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.7 Get SN

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appGetSNNumSync (CHAR sn)*

- **parameter**

  ***** sn： a buffer pointer to store returned sn

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.8 Get active CID information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetActedCidSync(UINT8 *cid, UINT8 *num)*

- **parameter**

  *****cid： CID in active state

  *****num： number of CID in active state

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.9 Get PSM setting information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetPSMSettingSync(UINT8 *psmmode, UINT32 *tauTime, UINT32 *activeTime)*

- **parameter**

  *psmmode： current PSM setting, details is shown below：

```
typedef enum CmiMmPsmReqModeEnum_Tag
{
    CMI_MM_DISABLE_PSM = 0,
    CMI_MM_ENABLE_PSM = 1,
    CMI_MM_DISCARD_PSM = 2 /*disable PSM, and discard PSM PARMs*/
}CmiMmPsmReqModeEnum;
```

  *tauTime： TAU time in second

  *activeTime： Active time in second

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.10 Get TAU information comes from network

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetTAUInfoSync(UINT32 *tauTime, UINT32 *activeTime)*

- **parameter**

  *tauTime： TAU time in second

  *activeTime：Active time in second

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.11 Set PSM information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appSetPSMSettingSync(UINT8 psmmode, UINT32 tauTime, UINT32 activeTime)

- **parameter**

  psmmode： PSM mode to be set， details is shown below：

```
typedef enum CmiMmPsmReqModeEnum_Tag
{
    CMI_MM_DISABLE_PSM = 0,
    CMI_MM_ENABLE_PSM = 1,
    CMI_MM_DISCARD_PSM = 2 /*disable PSM, and discard PSM PARMs*/
}CmiMmPsmReqModeEnum;
```

tauTime： TAU time in second

activeTime：Active time in second

- **return**

  NBStatusSuccess: success；NBStatusFail: fail；NBStatusTimeout: timeout

3.7.12 Get UE register state

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetCeregStateSync(UINT8 *state)*

- **parameter**

  * state：register state, details is shown below:

```
typedef enum CmiCeregStateEnum_Tag
{
    CMI_PS_NOT_REG = 0,
    CMI_PS_REG_HOME = 1,
    CMI_PS_NOT_REG_SEARCHING = 2,
    CMI_PS_REG_DENIED = 3,
    CMI_PS_REG_UNKNOWN = 4,
    CMI_PS_REG_ROAMING = 5,
    CMI_PS_REG_SMS_ONLY_HOME = 6,              // not applicable
    CMI_PS_REG_SMS_ONLY_ROAMING = 7,           // not applicable
    CMI_PS_REG_EMERGENCY = 8,
    CMI_PS_REG_CSFB_NOT_PREFER_HOME = 9, // not applicable
    CMI_PS_REG_CSFB_NOT_PREFER_ROAMING = 10 //not applicable
}CmiCeregStateEnum;
```

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.13 Get UE Netinfo

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetNetInfoSync(UINT32 cid, NmAtiSyncResult *result)*

- **parameter**

cid： CID to be checked

*result：returned Net information

For details please refer to NmNetifStatus below：

```
Typedef enum NmNetifStatus_Tag
{
    /*
    * no netif created, and UE is not under dialing
    */
    NM_NO_NETIF_NOT_DIAL,
    /*
    * no netif created
    * 1> maybe UE is not registerd, but already under dialing,
          and NETIF maybe be OK later.
      2> or APP request one specific NETIF(by CID), which not created
      3> when a NETIF deactivated
    */
    NM_NO_NETIF_OR_DEACTIVATED,
    /*
    * netif is created, but NETIF is OOS
    */
    NM_NETIF_OOS,
    /*
    * netif is suspend
      * When NETIF is OOS, must also be suspend, in such case, state should be set
      * to OOS
    */
    NM_NETIF_SUSPEND,
    /*
    * netif is activated
    */
    NM_NETIF_ACTIVATED,
    /*
    * netif is activated, but some info changed, such as:
    * 1> ipv4v6 type, ipv6 RS success later, need to report NETIF changed;
    * 2> ipv4v6 two bearers, if one bearer deactivated, but another is still exist,
          Also need to report NETIF changed;
    * 3> This status only used for event indication report;
    */
    NM_NETIF_ACTIVATED_INFO_CHANGED
}NmNetifStatus;
```

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.14 Get Location information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetLocationInfoSync(UINT16 *tac, UINT32 *cellId)*

- **parameter**

  *tac： TAC value

  *cellID：cell ID

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.15 Get UE EDRX setting

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetEDRXSettingSync(UINT8 *actType, UINT32 *nwEdrxValueMs, UINT32 *nwPtwMs)*

- **parameter**

  *actType： returned act type;

  0 means EDRX is not enabled，5 means EDRC is enabled

  *nwEdrxValueMs： EDRX value in ms

  *nwPtwMs： paging time window value in ms

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.16 Set UE EDRX information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appSetEDRXSettingSync(UINT8 modeVal, UINT8 actType, UINT32 reqEdrxValueMs)

- **parameter**

  modeVal： mode value to be set, details is shown below：

```
typedef enum CmiMmEdrxModeEnum_Tag
{
    CMI_MM_DISABLE_EDRX = 0,
    CMI_MM_ENABLE_EDRX_AND_DISABLE_IND = 1,
    CMI_MM_ENABLE_EDRX_AND_ENABLE_IND = 2,
    CMI_MM_DISCARD_EDRX = 3        /*Disable the use of eDRX and discard all parameters for eDRX*/
}CmiMmEdrxModeEnum;
```

  actType： act type value to be set ， details is shown below：

```
typedef enum CmiMmEdrxActTypeEnum_Tag
{
    CMI_MM_EDRX_NO_ACT_OR_NOT_USE_EDRX = 0,
    CMI_MM_EDRX_EC_GSM_IOT = 1, //useless
```

```
    CMI_MM_EDRX_GSM = 2, //useless
    CMI_MM_EDRX_UMTS = 3, //useless
    CMI_MM_EDRX_LTE = 4, //useless
    CMI_MM_EDRX_NB_IOT = 5
}CmiMmEdrxActTypeEnum;
```

reqEdrxValueMs： request EDRX value in ms

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.17 Get PLMN search power level information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  UINT8 appGetSearchPowerLevelSync(void)

- **parameter**

  N/A

- **return**

  PLMN search power level，range {0，3}

3.7.18 Get APN setting information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appGetAPNSettingSync(CHAR vpn)*

- **parameter**

  vpn：buffer pointer to store returned vpn information，should not less than 30 bytes

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.19 Check NITZ sync information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appCheckSystemTimeSync(void)

- **parameter**

- **return**

  NBStatusSuccess system time is synced via NITZ；NBStatusFail system time is not synced via NITZ

3.7.20 Get system time in seconds

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appGetSystemTimeSecsSync(time_t time)*

- **parameter**

  time：seconds since 1970/1/1 00:00:00. If system time is not synced, the default value is started from 2000/1/1 00:00:00. System will update when NITZ is reported or get time via SNTP.

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.21 Get UTC time

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appGetSystemTimeUtcSync ***(utc_timer_value_t *time)***

- **parameter**

  time：returned UTC time， **utc_timer_value_t** is shown below:

```
typedef __PACKED_STRUCT _utc_timer_value
{
    // UTCtimer1        ---UINT16 year--UINT8 mon--UINT8 day        //
    // UTCtimer2        ---UINT8 hour--UINT8 mins--UINT8 sec--INT8 tz   //
    unsigned int UTCtimer1;
    unsigned int UTCtimer2;
    unsigned int UTCsecs;    //secs since 1970//
    unsigned int UTCms;      //current ms //
    unsigned int timeZone;
} utc_timer_value_t;
```

  If system time is not synced, the default value is started from 2000/1/1 00:00:00. System will update when NITZ is reported or get time via SNTP.

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.22 Set UTC time

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appSetSystemTimeUtcSync ***(UINT32 time1, UINT32 time2)***

- **parameter**

  time1 set yy/mm/dd, time2 set hh/mm/ss/tz

  // time1   ---UINT16 year--UINT8 mon--UINT8 day   //

  // time2   ---UINT8 hour--UINT8 mins--UINT8 sec--INT8 tz   //

- **return**

NBStatusSuccess: success；NBStatusFail: fail

3.7.23 Get UE CElevel information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  UINT8 appGetCELevelSync(void)

- **parameter**

  无

- **return**

  Coverage Class in the serving cell

  0 No Coverage Class in the serving cell

  1 Coverage Class 1

  2 Coverage Class 2

  3 Coverage Class 3

  4 Coverage Class 4

  5 Coverage Class 5

3.7.24 Get UE signal strength

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetSignalInfoSync(UINT8 *csq, INT8 *snr, INT8 *rsrp)*

- **parameter**

  *csq： retuned csq;

  *snr： retuned snr;

  *rsrp：retuned rsrp;

- **return**

  csq us signal strength , range is 0-31, bigger is better. 99 means not get signal strength

  snr signal noise ratio，bigger is better

  rsrp rsrp information， 127 means invalid rsrp.

3.7.25 Get UE version information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  UINT8 appGetNBVersionInfo (void)*

- **parameter**

  N/A

- **return**

  return SDK version information

3.7.26 Set CFUN

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appSetCFUN(UINT8 fun)

- **parameter**

  fun: 0 means disable modem，1 means enable modem.

  For details refer to AT+CFUN

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.27 Set UE default boot CFUN mode

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appSetBootCFUNMode(UINT8 mode)

- **parameter**

  mode: 0 means disable modem，1 enable modem

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.28 Get UE default boot CFUN mode

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  UINT8 appGetBootCFUNMode(void)

- **parameter**

  N/A

- **return**

  0 means default boot mode is CFUN0；1 means default boot mode is CFUN1；0Xff means fail

3.7.29 Open SIM logic channel

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appSetSimLogicalChannelOpenSync(UINT8 *sessionID, UINT8 *dfName)*

- **parameter**

  SessionID: Pointer to a new logical channel number returned by SIM

  dfName: Pointer to DFname selected on the new logical channel

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.30 Close SIM logic channel

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  CmsRetId appSetSimLogicalChannelCloseSync(UINT8 sessionID)

- **parameter**

  SessionID: the logical channel number to be closed

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.31 Generic SIM logical channel access

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appSetSimGenLogicalChannelAccessSync(UINT8 sessionID, UINT8 *command, UINT8 cmdLen, UINT8 *response, UINT8 *respLen)*

- **parameter**

  sessionID: logical channel number

  command: Pointer to command apdu

  cmdLen: the length of command apdu

  response: Pointer to response apdu

  respLen：the length of command apdu

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.32 Generic SIM access

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appSetRestrictedSimAccessSync(CrsmCmdParam *pCmdParam, CrsmRspParam *pRspParam)*

- **parameter**

  pCmdParam: Pointer to command parameters

  pRspParam: Pointer to response parameters

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.33 BAND setting

- **header file**

  #include "ps_lib_api.h"

- **prototype**

*CmsRetId appSetBandModeSync(UINT8 networkMode, UINT8 bandNum, UINT8 *orderBand)*

- **parameter**

  networkMode: network mode to be set

  bandNum: number of bands to be set

  orderBand：bands to be set in order

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.34 Get BAND setting

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetBandModeSync(UINT8 *networkMode, UINT8 *bandNum, UINT8 *orderBand)*

- **parameter**

  networkMode: pointer to returned network mode

  bandNum: pointer to returned number of band

  orderBand：pointer to returned band

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.35 Get supported BAND

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetSupportedBandModeSync (UINT8 *networkMode, UINT8 *bandNum, UINT8 *orderBand)*

- **parameter**

  networkMode: pointer to returned network mode

  bandNum: pointer to returned number of band

  orderBand：pointer to returned band

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.36 Get basic cell information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId appGetECBCInfoSync(BasicCellListInfo *bcListInfo)*

- **parameter**

bcListInfo: pointer to returned serving cell and neighbor cell

details of BasicCellListInfo is shown below

```
#define NCELL_INFO_CELL_NUM        4
typedef struct BasicCellListInfoTag
{
    BOOL                    sCellPresent;
    UINT8                   nCellNum;
    SrvCellBasicInfo        sCellInfo;
    NCellBasicInfo          nCellList[NCELL_INFO_CELL_NUM];
}BasicCellListInfo;

typedef struct SrvCellBasicInfoTag
{
    UINT16    mcc;
    UINT16    mncWithAddInfo;//if 2-digit MNC type, the 4 MSB bits should set to 'F'
    /* Example:
    *    46000; mcc = 0x0460, mnc = 0xf000
    *    00101; mcc = 0x0001, mnc = 0xf001
    *    46012; mcc = 0x0460, mnc = 0xf012
    *    460123; mcc = 0x0460, mnc = 0x0123
    */

     //DL earfcn (anchor earfcn), range 0 - 262143
     UINT32              earfcn;
     //the 28 bits Cell-Identity in SIB1, range 0 - 268435455
     UINT32              cellId;

     //physical cell ID, range 0 - 503
     UINT16              phyCellId;
     // value in dB, value range: -30 ~ 30
     BOOL                snrPresent;
     INT8                snr;

        //value in units of dBm, value range: -156 ~ -44
        INT16               rsrp;
        //value in units of dB, value range: -34 ~ 25
        INT16               rsrq;
    }SrvCellBasicInfo;

    typedef struct NCellBasicInfoTag
    {
        UINT32              earfcn;       //DL earfcn (anchor earfcn), range 0 - 262143

        UINT16              phyCellId;
```

```
    UINT16          revd0;

    //value in units of dBm, value range: -156 ~ -44
    INT16           rsrp;
    //value in units of dB, value range: -34 ~ 25
    INT16           rsrq;
}NCellBasicInfo;
```

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.37 Set cell/frequency lock/unlock

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId CmsRetId appSetCiotFreqSync(CiotFreqParams *pCiotFreqParams)*

- **parameter**

  pCiotFreqParams: options to be Set,

```
typedef struct CiotFreqParams_Tag
    {
    UINT8   mode;       // CmiDevGetFreqModeEnum   3:means UE has set preferred EARFCN list and has locked EARFCN
    UINT8   cellPresent;// indicate whether phyCellId present

    UINT16  phyCellId; // phyCell ID, 0 - 503

    UINT8   arfcnNum; // 0 is not allowed for mode is CMI_DEV_SET_PREFER_FREQ (1);
                      // max number is CMI_DEV_SUPPORT_MAX_FREQ_NUM
    UINT8   reserved0;
    UINT16  reserved1;
    UINT32  lockedArfcn;//locked EARFCN
    UINT32  arfcnList[SUPPORT_MAX_FREQ_NUM];
}CiotFreqParams;
```

- **return**

  NBStatusSuccess: success；NBStatusFail: fail

3.7.38 Get cell/frequency lock/unlock information

- **header file**

  #include "ps_lib_api.h"

- **prototype**

  *CmsRetId CmsRetId appGetCiotFreqSync(CiotFreqParams *pCiotFreqParams)*

- **parameter**

pCiotFreqParams: returned parameter，refer to CiotFreqParams

- **return**

NBStatusSuccess: success；NBStatusFail: fail

4. Appendix

4.1 Power up sequence

The back item is performed automatically in SDK, the red item is interaction with developer. Developer could get information and control UE via URC event or API.

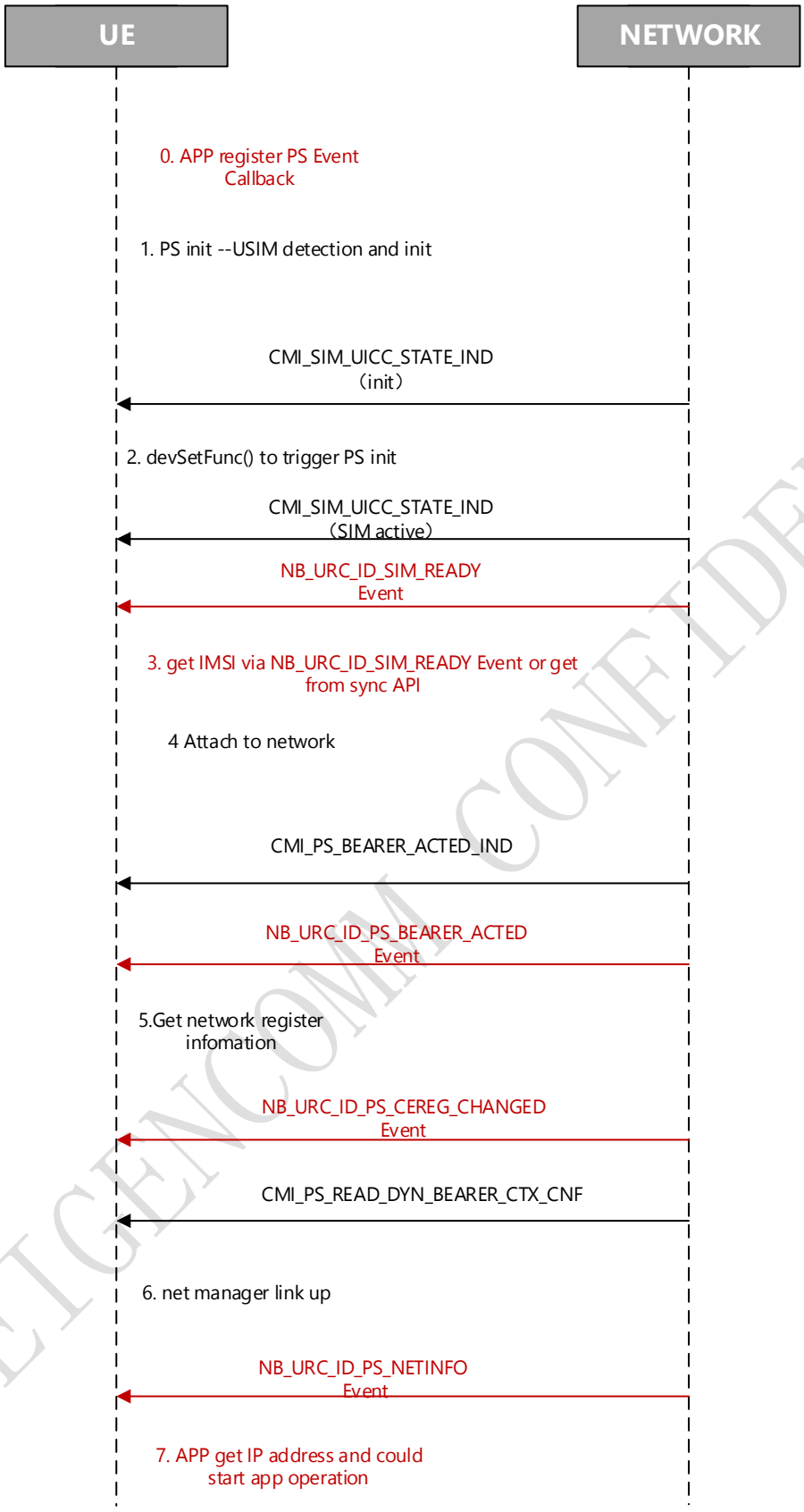

Figure 3 power up sequence

4.2 OpenCPU demo example

Demo examples are provided in SDK(project/$(TARGET)/apps) to give reference for OpenCPU developer.

Example	description
coap_example	Coap example
driver_example	Driver API usage example
fs_example	Littlefs operation example
https_example	https example
lwm2m_example	wakaama lwm2m example
mqtt_example	MQTT example
onenet_example	CMCC onenet access example
oc_example	CTCC OceanConnect access example
socket_example	Tcp/udp client/server example

5. Version

Version	Date	Comments
Draft	2018-11-11	Draft
V1.0	2020-04-13	English version
V1.1	2020-08-12	Update user reserved RAM size and usage